

FINAL DATA BASE PROJECT

ACCESS

OZAN KURBAN

TASK 1:Entity Relationship Diagram (ERD)

Demonstrate your understanding of database structure and design by creating a detailed ERD that accurately represents your grading database.

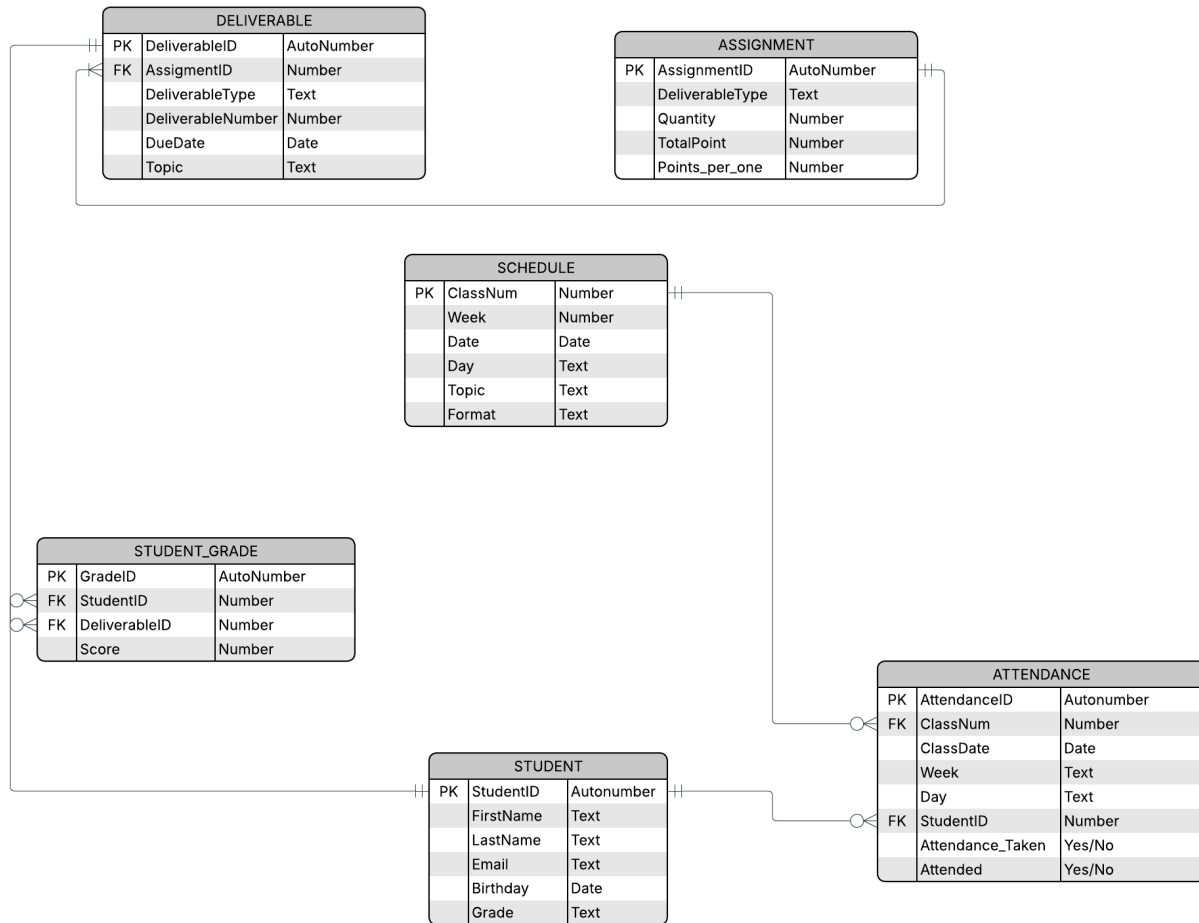
Instructions

- Use Lucidchart to create your ERD.
- Include:
 - All tables in your grading database
 - Primary keys and data types
 - Relationships between tables
 - Cardinality and participation constraints (make sure to include minimum and maximum)

Submission Format

Export your ERD as an image (PNG or JPEG). Paste it directly into your PDF under Task 1, labeled as:

Figure 1. Entity Relationship Diagram of the Grading Database.



TASK2:SQL Table Creation Code

Translate your conceptual database design into SQL schema code.

Instructions

- Use Lucidchart's export function (you can use AI to modify the SQL syntax to MS Access)
- Include table names, fields, data types, keys, and constraints.

Original Code:

```
CREATE TABLE `STUDENT` (
  `StudentID` Autonumber,
  `FirstName` Text,
```

```
`LastName` Text,  
`Email` Text,  
`Birthday` Date,  
`Grade` Text,  
PRIMARY KEY (`StudentID`)  
);
```

```
CREATE TABLE `SCHEDULE` (  
  `ClassNum` Number,  
  `Week` Number,  
  `Date` Date,  
  `Day` Text,  
  `Topic` Text,  
  `Format` Text,  
  PRIMARY KEY (`ClassNum`)  
);
```

```
CREATE TABLE `ATTENDANCE` (  
  `AttendancelD` Autonumber,  
  `ClassNum` Number,  
  `ClassDate` Date,  
  `Week` Text,  
  `Day` Text,  
  `StudentID` Number,  
  `Attendance_Taken` Yes/No,  
  `Attended` Yes/No,  
  PRIMARY KEY (`AttendancelD`),
```

```
FOREIGN KEY (`StudentID`)
    REFERENCES `STUDENT`(`StudentID`),
FOREIGN KEY (`ClassNum`)
    REFERENCES `SCHEDULE`(`ClassNum`)
);
```

```
CREATE TABLE `ASSIGNMENT` (
    `AssignmentID` AutoNumber,
    `DeliverableType` Text,
    `Quantity` Number,
    `TotalPoint` Number,
    `Points_per_one` Calculated,
    PRIMARY KEY (`AssignmentID`)
);
```

```
CREATE TABLE `DELIVERABLE` (
    `DeliverableID` AutoNumber,
    `AssignmentID` Number,
    `DeliverableType` Text,
    `DeliverableNumber` Number,
    `DueDate` Date,
    `Topic` Text,
    PRIMARY KEY (`DeliverableID`),
    FOREIGN KEY (`AssignmentID`)
        REFERENCES `ASSIGNMENT`(`AssignmentID`)
);
```

```

CREATE TABLE `STUDENT_GRADE` (
  `GradeID` AutoNumber,
  `StudentID` Number,
  `DeliverableID` Number,
  `Score` Number,
  PRIMARY KEY (`GradeID`),
  FOREIGN KEY (`StudentID`)
    REFERENCES `STUDENT`(`StudentID`),
  FOREIGN KEY (`DeliverableID`)
    REFERENCES `DELIVERABLE`(`DeliverableID`)
);

```

Optimized Access SQL:

```

CREATE TABLE `STUDENT` (
  `StudentID` COUNTER,
  `FirstName` VARCHAR(50),
  `LastName` VARCHAR(50),
  `Email` VARCHAR(100),
  `Birthday` DATETIME,
  `Grade` VARCHAR(10),
  CONSTRAINT PK_STUDENT PRIMARY KEY (`StudentID`)
);

```

```

CREATE TABLE `SCHEDULE` (
  `ClassNum` INTEGER,
  `Week` INTEGER,
  `Date` DATETIME,
  `Day` VARCHAR(15),

```

```
`Topic` VARCHAR(255),  
`Format` VARCHAR(50),  
CONSTRAINT PK_SCHEDULE PRIMARY KEY (`ClassNum`)  
);
```

```
CREATE TABLE `ATTENDANCE` (  
  `AttendanceID` COUNTER,  
  `ClassNum` INTEGER,  
  `ClassDate` DATETIME,  
  `Week` VARCHAR(50),  
  `Day` VARCHAR(20),  
  `StudentID` INTEGER,  
  `Attendance_Taken` YESNO,  
  `Attended` YESNO,  
  CONSTRAINT PK_ATTENDANCE PRIMARY KEY (`AttendanceID`),  
  CONSTRAINT FK_ATT_STUDENT FOREIGN KEY (`StudentID`) REFERENCES  
  `STUDENT`(`StudentID`),  
  CONSTRAINT FK_ATT_SCHEDULE FOREIGN KEY (`ClassNum`) REFERENCES  
  `SCHEDULE`(`ClassNum`)  
);
```

```
CREATE TABLE `ASSIGNMENT` (  
  `AssignmentID` COUNTER,  
  `DeliverableType` VARCHAR(50),  
  `Quantity` INTEGER,  
  `TotalPoint` DOUBLE,  
  `Points_per_one` DOUBLE, -- Sayı (Ondalıkli olabilir diye DOUBLE seçtik)  
  CONSTRAINT PK_ASSIGNMENT PRIMARY KEY (`AssignmentID`)  
);
```

```

CREATE TABLE `DELIVERABLE` (
  `DeliverableID` COUNTER,
  `AssignmentID` INTEGER,
  `DeliverableType` VARCHAR(50),
  `DeliverableNumber` INTEGER,
  `DueDate` DATETIME,
  `Topic` VARCHAR(255),
  CONSTRAINT PK_DELIVERABLE PRIMARY KEY (`DeliverableID`),
  CONSTRAINT FK_DELIV_ASSIGN FOREIGN KEY (`AssignmentID`)
REFERENCES `ASSIGNMENT`(`AssignmentID`)
);

```

```

CREATE TABLE `STUDENT_GRADE` (
  `GradeID` COUNTER,
  `StudentID` INTEGER,
  `DeliverableID` INTEGER,
  `Score` DOUBLE,
  CONSTRAINT PK_STUDENT_GRADE PRIMARY KEY (`GradeID`),
  CONSTRAINT FK_GRADE_STUDENT FOREIGN KEY (`StudentID`)
REFERENCES `STUDENT`(`StudentID`),
  CONSTRAINT FK_GRADE_DELIV FOREIGN KEY (`DeliverableID`)
REFERENCES `DELIVERABLE`(`DeliverableID`)
);

```

TASK 3: Attendance Query To Date

Aggregate and calculate student attendance percentage up to date . Do not include classes that were canceled, only classes where attendance was recorded.

Instructions

Create a query that outputs:

- Student Name

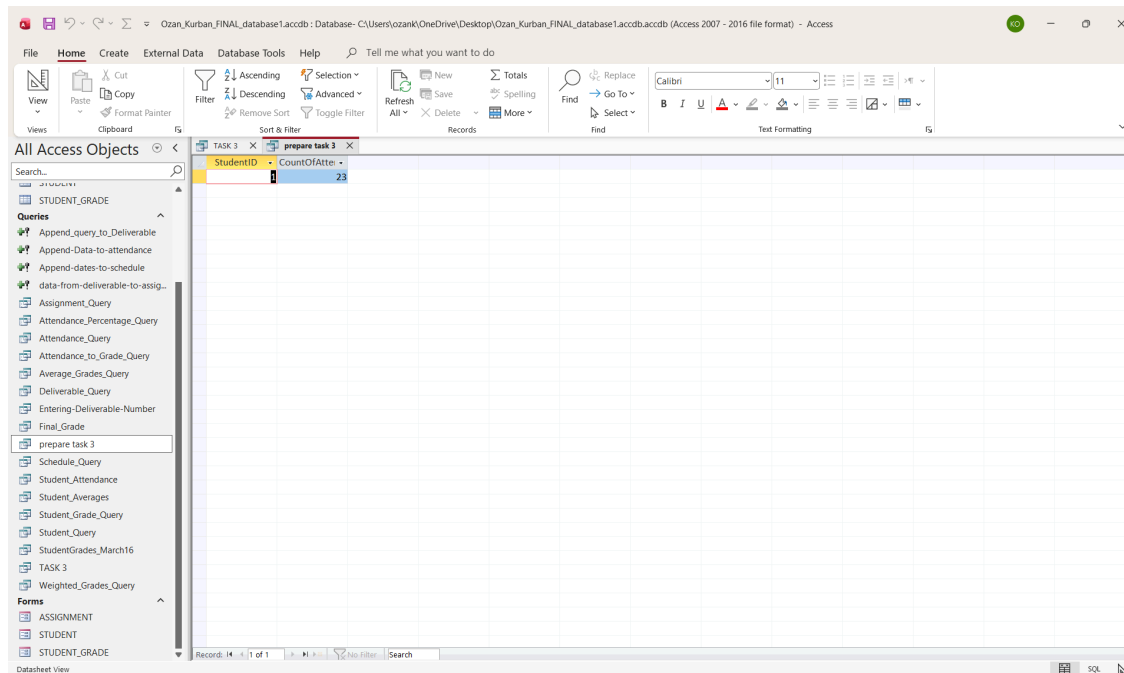
- Number of class attended
- Number of class not attended
- Attendance percentage
- Weighted Points (Max)
- Weighted Points Earned = (Attendance percentage × Weight)

Submission

- Part A – SQL Code (as text)
- Part B – Query Results Table (formatted)

For calculation of attendance percentage I found count of attendance

```
SELECT
    ATTENDANCE.StudentID,
    Count(ATTENDANCE.Attendance_Taken) AS CountOfAttendance_Taken
FROM
    SCHEDULE
    INNER JOIN ATTENDANCE ON SCHEDULE.ClassNum = ATTENDANCE.ClassNum
WHERE
    (((ATTENDANCE.Attendance_Taken) = Yes))
GROUP BY
    ATTENDANCE.StudentID
HAVING
    (((ATTENDANCE.StudentID) = 1));
```

SELECT

```

S.FirstName & " " & S.LastName AS StudentName,
SUM(IIf(A.Attended = 'Yes', 1, 0)) AS ClassesAttended,
SUM(IIf(A.Attended = 'No', 1, 0)) AS ClassesNotAttended,
ROUND(
    (
        SUM(IIf(A.Attended = 'Yes', 1, 0)) * 1.0 / SUM(IIf(A.Attendance_Taken = 'Yes', 1,
0))
    ) * 100,
    2
) AS AttendancePercent,
DLookup (
    "TotalPoint",
    "ASSIGNMENT",
    "DeliverableType='Attendance'"
) AS WeightedPointsMax,

```

```

ROUND(
    (
        (
            SUM(If(A.Attended = 'Yes', 1, 0)) * 1.0 / SUM(If(A.Attendance_Taken = 'Yes',
1, 0))
        )
    ) * DLookup (
        "TotalPoint",
        "ASSIGNMENT",
        "DeliverableType='Attendance'"
    ),
    2
) AS WeightedPointsEarned
FROM
    STUDENT AS S
    INNER JOIN ATTENDANCE AS A ON S.StudentID = A.StudentID
WHERE
    A.Attendance_Taken = 'Yes'
    AND A.ClassDate <= #5 / 1 / 2026 #
GROUP BY
    S.FirstName,
    S.LastName
ORDER BY
    S.LastName,
    S.FirstName;

```

StudentName	ClassesAttended	ClassesNotAttended	AttendancePercent	WeightedPointsMax	WeightedPointsEarned
Ada Kurbani	23	0	100	6	6
Ozan Kurbani	23	0	100	6	6

TASK 4: Append Attendance Score to the STUDENT_GRADE table

Once you calculate the student attendance percentages up to date (Tip - use 4/30/2026 as your cut-off date), append them to the STUDENT_GRADE table so they will be counted towards the final grade calculation.

Instructions

Create a query that appends:

- StudentID
- DeliverableID (hint -- from the weight table take the primary key for Participation or Attendance)
- Grade or score

Submission

- Part A – SQL Code (as text)
- Part B – Query Results Table (formatted)
- **INSERT INTO** STUDENT_GRADE (StudentID, DeliverableID, Score)
- **SELECT**
- S.StudentID,

- DLookup("AssignmentID","ASSIGNMENT","DeliverableType='Attendance'") AS DeliverableID,
-
- ROUND(
- (SUM(IIf(A.Attended='Yes',1,0)) * 1.0 /
- SUM(IIf(A.Attendance_Taken='Yes',1,0))) * 100
- , 2) AS AttendancePercentScore
-
- FROM
- STUDENT AS S
- INNER JOIN ATTENDANCE AS A
- ON S.StudentID = A.StudentID
-
- WHERE
- A.Attendance_Taken='Yes'
- AND A.ClassDate <= #4/30/2026#
-
- GROUP BY
- S.StudentID;
-

Ozan_Kurban_FINAL_database1.accdb - Database- C:\Users\ozank\OneDrive\Desktop\Ozan_Kurban_FINAL_database1.accdb (Access 2007 - 2016 file format) - Access

File Home Create External Data Database Tools Help Table Fields Table Tell me what you want to do

Compact and Repair Database Visual Basic Run Macro Relationships Object Dependencies Database Documenter Analyze Performance Access Database SharePoint Database Add-ins~

All Access Objects

Search...

Tables

- ASSIGNMENT
- ATTENDANCE
- DELIVERABLE
- LETTER_GRADE
- SCHEDULE
- STUDENT
- STUDENT_GRADE

Queries

- Append_query_to_Deliverable
- Append-Data-to-attendance
- Append-dates-to-schedule
- data-from-deliverable-to-assig...
- TASK 4
- Assignment_Query
- Attendance_Percentage_Query
- Attendance_Query
- Attendance_to_Grade_Query
- Average_Grades_Query
- Deliverable_Query
- Entering-Deliverable-Number
- Final_Grade
- Schedule_Query
- Student_Attendance
- Student_Averages
- Student_Grade_Query
- Student_Query

STUDENT_GRADE

GradeID	StudentID	DeliverableID	Score	Click to Add
1	1	1	0	
2	2	1	0	
3	1	2	0	
4	1	3	100	
5	1	4	100	
6	1	5	100	
7	1	6	100	
8	1	7	90	
9	1	8	90	
10	1	9	100	
11	1	10	98	
12	1	11	100	
13	1	12	0	
14	1	13	100	
15	1	14	98	
16	1	15	100	
17	1	16	100	
18	1	17	0	
19	1	18	0	
20	1	19	100	
21	1	20	100	
22	1	21	100	
23	1	22	100	
24	1	11	100	
25	2	11	100	
(New)	0	0	0	

Records: 14 of 1 of 25 No Filter Search

Worksheet View

Deliverable Summary Query To Date

Aggregate and calculate student performance across multiple deliverable types up to date

Instructions

Create a query that outputs:

- **Student Name**
- **Deliverable Type**
- **Quantity completed**
- **Average Score (using Date() for current cutoff)**
- **Weighted Points (Max)**
- **Weighted Points Earned = (Average × Weight)**

Use appropriate JOIN and GROUP BY statements.

Submission

- **Part A – SQL Code (as text)**
- **Part B – Query Results Table (formatted)**

SELECT

```
S.FirstName & " " & S.LastName AS StudentName,  
A2.DeliverableType,  
COUNT(SG.Score) AS QuantityCompleted,  
ROUND(AVG(SG.Score), 2) AS AverageScore,  
A2.TotalPoint AS WeightedPointsMax,  
ROUND( (AVG(SG.Score)/100) * A2.TotalPoint, 2 ) AS WeightedPointsEarned
```

FROM

```
(STUDENT AS S
INNER JOIN STUDENT_GRADE AS SG
    ON S.StudentID = SG.StudentID)
INNER JOIN ASSIGNMENT AS A2
    ON SG.DeliverableID = A2.AssignmentID
```

WHERE

```
SG.Score IS NOT NULL
AND (
    SELECT MAX(D.DueDate)
    FROM DELIVERABLE AS D
    WHERE D.AssignmentID = A2.AssignmentID
) <= #4/30/2026#
```

GROUP BY

```
S.FirstName,
S.LastName,
A2.DeliverableType,
A2.TotalPoint
```

ORDER BY

```
S.LastName, S.FirstName, A2.DeliverableType;
```

Microsoft Access window: Ozan_Kurban_FINAL_database1.accdb : Database- C:\Users\ozank\OneDrive\Desktop\Ozan_Kurban_FINAL_database1.accdb (Access 2007 - 2016 file format) - Access

Table: TASK 5

StudentName	DeliverableType	QuantityCompleted	AverageScore	WeightedPointsMax	WeightedPointsEarned
Ozan Kurban	Capstone Project	1	0	4	0
Ozan Kurban	Capstone Project	1	0	4	0
Ozan Kurban	Exercise	1	0	12	0
Ozan Kurban	Final Project	1	90	12	10.8
Ozan Kurban	Midterm Project	1	90	10	9
Ozan Kurban	Quiz	1	100	14	14
Ozan Kurban	SAM Project	1	100	12	12
Ozan Kurban	Simulation Training	1	100	5	5
Ozan Kurban	SQL Tutorial Exercise	1	100	3	3

Records: 14 of 9

TASK 5:Final Grade Calculation Query (Up to Date)

Compute cumulative performance across deliverable types up to a date (May 1).

Instructions

- Use results from Task 3 as a base.
- Calculate each student's final grade as the sum of weighted points earned.
- Format to two decimals.
- Include student names.

Submission

- Part A – SQL Code
- Part B – Final Grade Results Table
- Hint - make sure that you calculate the points you received up to date as a percentage of the total points available. It should not include the final test and project

SELECT

S.FirstName & " " & S.LastName AS StudentName,

ROUND(

SUM(

/* Weighted Points Earned for each deliverable type */

(SG.Score / 100.0) * A.TotalPoint

)

, 2) AS FinalGrade

FROM

(STUDENT AS S

INNER JOIN STUDENT_GRADE AS SG

ON S.StudentID = SG.StudentID)

INNER JOIN ASSIGNMENT AS A

ON SG.DeliverableID = A.AssignmentID

WHERE

/* Final Test ve Final Project dahil edilmez */

A.DeliverableType NOT IN ('Final Test', 'Final Project')

/* Cutoff: only deliverables with due dates up to May 1 */

AND SG.DeliverableID IN (

SELECT D.DeliverableID

FROM DELIVERABLE AS D

WHERE D.DueDate <= #5/1/2026#

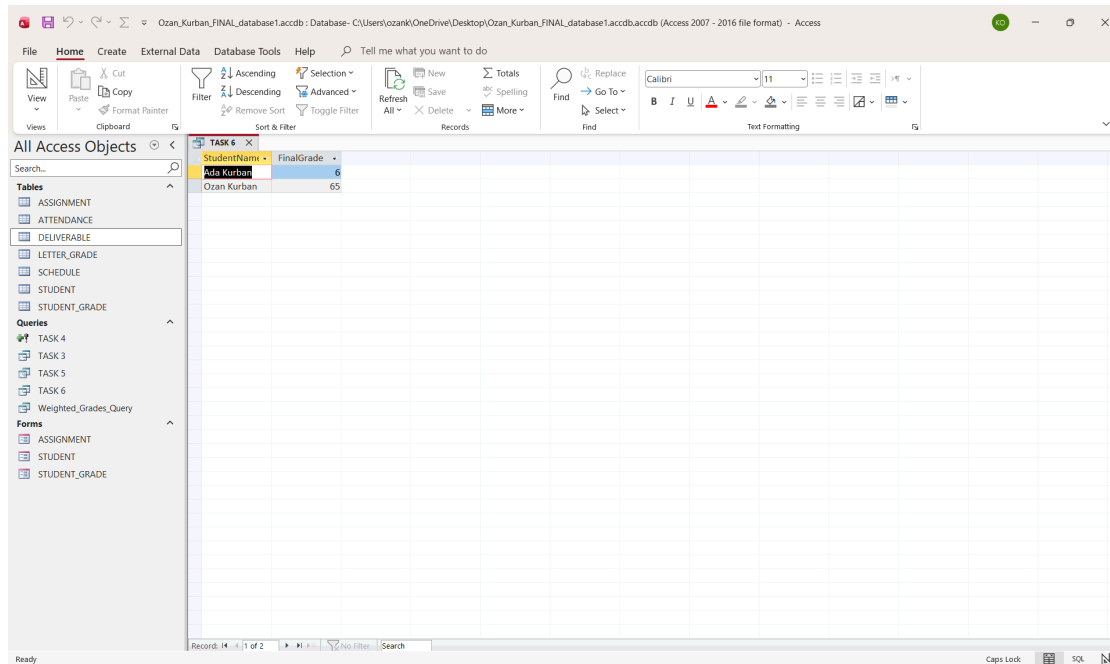
)

GROUP BY

S.FirstName, S.LastName

ORDER BY

S.LastName, S.FirstName;



Letter Grade Calculation Queries

Map numeric grades to letter grades using a lookup table. Do it for your grade up to date.

Instructions

1. Create a LETTER_GRADE_SCALE table:
 - MinScore, MaxScore, LetterGrade
2. Populate it based on your grading scale.
3. Join it to your final grade query using BETWEEN.

Submission

- Part A – SQL Code (join logic)
- Part B – Table of numeric and letter grades

```

SELECT
    FG.StudentName,
    FG.FGScore,
    L.Grade AS LetterGrade
FROM
    (
        SELECT
            S.FirstName & ' ' & S.LastName AS StudentName,
            CDBl (ROUND(SUM((SG.Score / 100.0) * A.TotalPoint), 2)) AS FGScore
        FROM
            (
                STUDENT AS S
                INNER JOIN STUDENT_GRADE AS SG ON S.StudentID = SG.StudentID
            )
            INNER JOIN ASSIGNMENT AS A ON SG.DeliverableID = A.AssignmentID
        WHERE
            A.DeliverableType NOT IN ('Final Test', 'Final Project')
            AND SG.DeliverableID IN (
                SELECT
                    DeliverableID
                FROM
                    DELIVERABLE
                WHERE
                    DueDate <= #5 / 1 / 2026 #
            )
        GROUP BY
            S.FirstName,
            S.LastName
    ) AS FG
    INNER JOIN LETTER_GRADE AS L ON FG.FGScore >= L.Min_Grade
    AND FG.FGScore <= L.Max_Grade
ORDER BY
    FG.StudentName

```

Minimum Grade Analysis

Estimate the lowest possible final grade if all missing work scores are zero (all ungraded work, like the final test and final project).

Instructions

- Replace all missing grades with 0.
- Include all deliverable types.
- Sum weighted scores with zeros.
- Output: Student Name and Minimum Possible Final Grade (2 decimals)

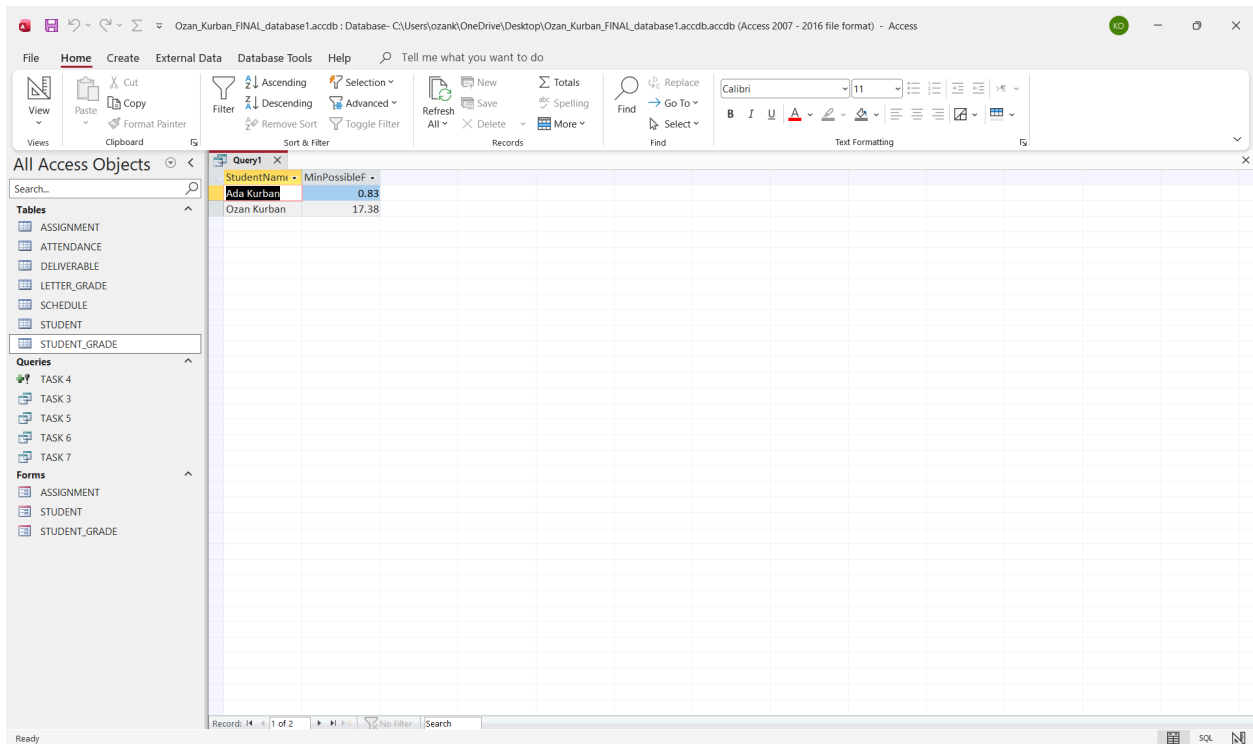
Submission

- Part A – SQL Code
- Part B – Results Table

```

SELECT
  S.FirstName & ' ' & S.LastName AS StudentName,
  ROUND(
    SUM(Nz(SG.Score, 0) * A.Points_per_one) / (
      SELECT
        Max(Max_Grade)
      FROM
        LETTER_GRADE
    ),
    2
  ) AS MinPossibleFinalGrade
FROM
  (
    (
      STUDENT AS S
      LEFT JOIN STUDENT_GRADE AS SG ON S.StudentID = SG.StudentID
    )
    LEFT JOIN DELIVERABLE AS D ON SG.DeliverableID = D.DeliverableID
  )
  LEFT JOIN ASSIGNMENT AS A ON D.AssignmentID = A.AssignmentID
GROUP BY
  S.FirstName,
  S.LastName;

```



TASK 6:Maximum Grade Analysis

Estimate the best possible final grade if all pending deliverables receive a perfect score.

Instructions

- Assign 100 for all ungraded deliverables.
- Reuse Final Grade query logic.
- Compute each student's maximum possible final grade.

Output

Student Name Maximum Possible Final Grade

Submission

- Part A – SQL Code
- Part B – Results Table

```
SELECT
    S.FirstName & ' ' & S.LastName AS StudentName,
    ROUND(
        SUM(
            (
```

The screenshot shows the Microsoft Access application window. The title bar indicates the file path: "Oran_Kurban_FINAL_database1.accdb : Database - C:\Users\ozank\OneDrive\Desktop\Oran_Kurban_FINAL_database1.accdb (Access 2007 - 2016 file format) - Access". The ribbon includes tabs for "Home", "Create", "External Data", "Database Tools", and "Help". The "Filter" button is active, and the "Sort & Filter" group is visible. The table "TASK 9" is displayed with the following data:

StudentID	MaxPosibled
Oran Kurban	1.89
Oran Kurban	23.29

The status bar at the bottom indicates "Record: 14 of 2".

TASK 7: Macro Development and Automation

Develop macros to automate grade calculation and report generation.

Instructions

Macro 1: Calculate Final Grades & Letter Grades

- Purpose: Automates calculation and update of all grades.
- Functionality:
 - Runs Final Grade and Letter Grade queries.
 - Refreshes all relevant tables.
 - Ensures results stay current with one click.
- Name: Calculate_Final_And_Letter_Grades

Macro 2: Generate Grade Reports

- Purpose: Automates export of updated grade summaries.
- Functionality:
 - Calculates your MAXIMUM final grade.
 - Calculate your corresponding letter grade
 - Generate a report with all students' names, final grades and letter grades.
- Enhancement: Enables instant professional report generation.

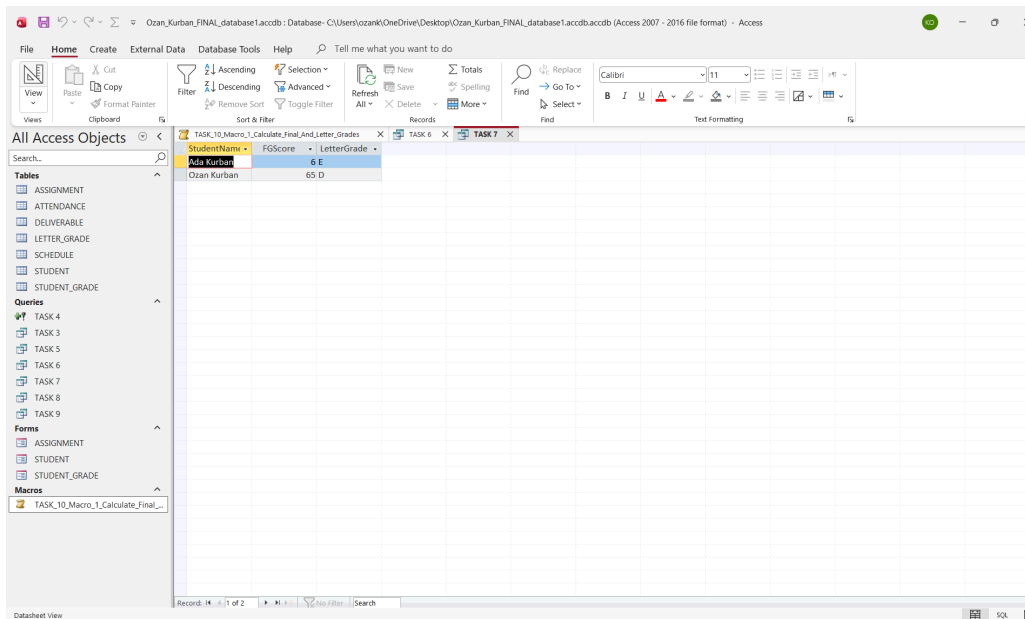
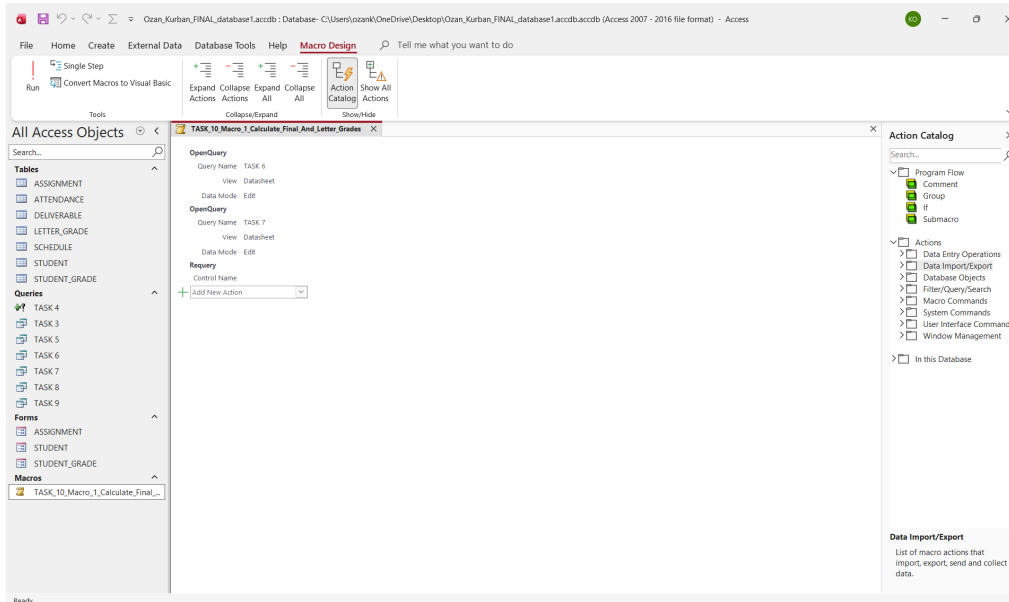
Submission

- Part A – Macro Descriptions (2–3 lines each)
- Part B – Macro Designer Screenshots
- Part C – Outcome Screenshots (macros in action)

Macro Descriptions

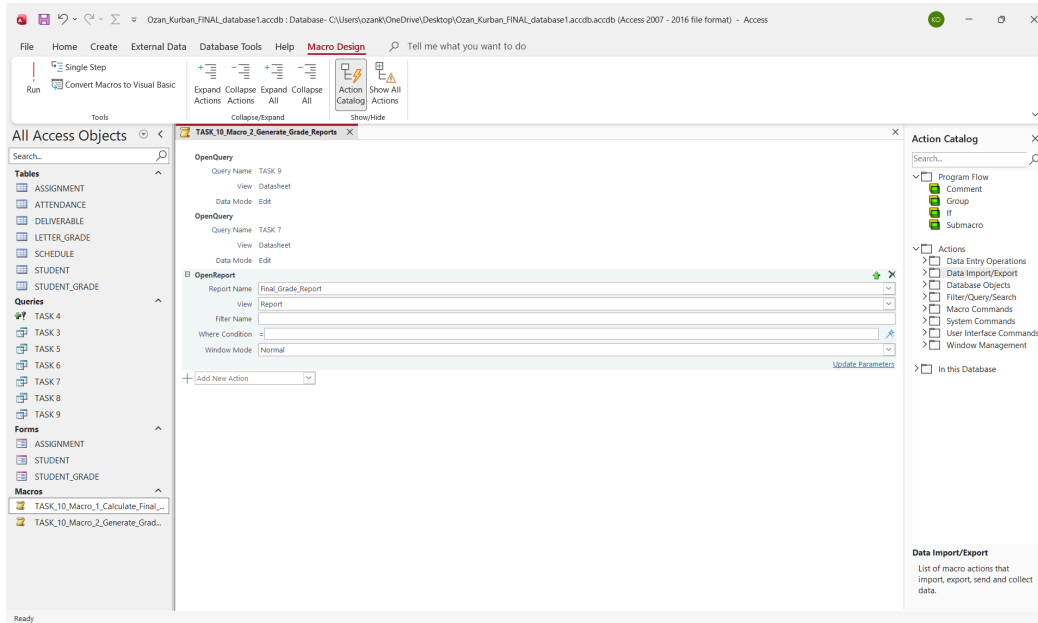
Macro 1: Calculate_Final_And_Letter_Grades

This macro automatically updates all students' final grades and letter grades. It runs the Final Grade and Letter Grade queries in sequence and refreshes the related tables to ensure that all grade calculations remain current. With one click, all grade results are recalculated and updated.



Macro 2: Generate_Grade_Reports

This macro calculates each student's maximum possible final grade and determines the corresponding letter grade. It then generates a report containing all students' names, final grades, and letter grades, and exports the updated report automatically. The macro enables quick, accurate, and professional report generation.



For create a report:

SELECT

T6.StudentName,

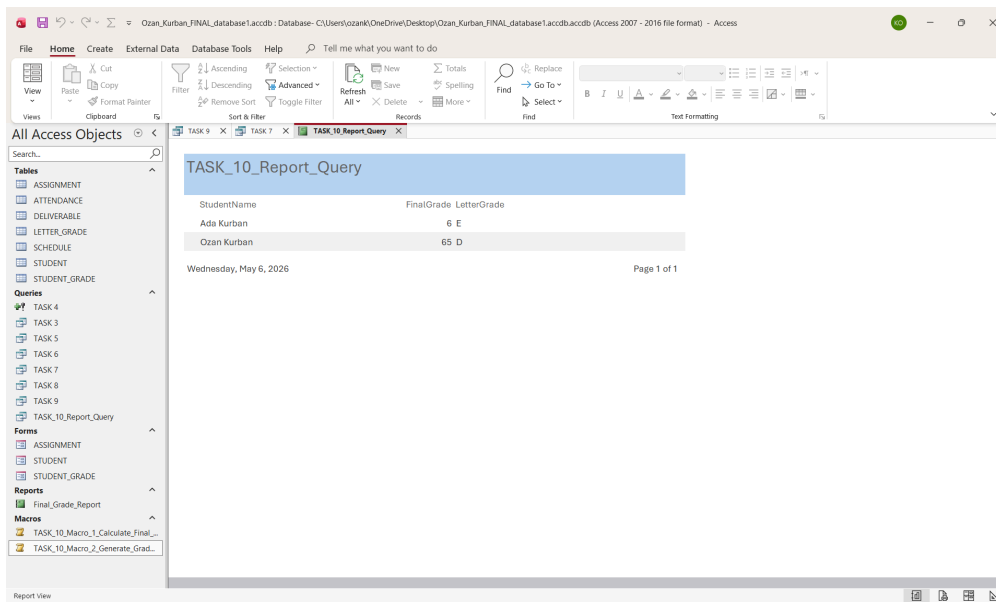
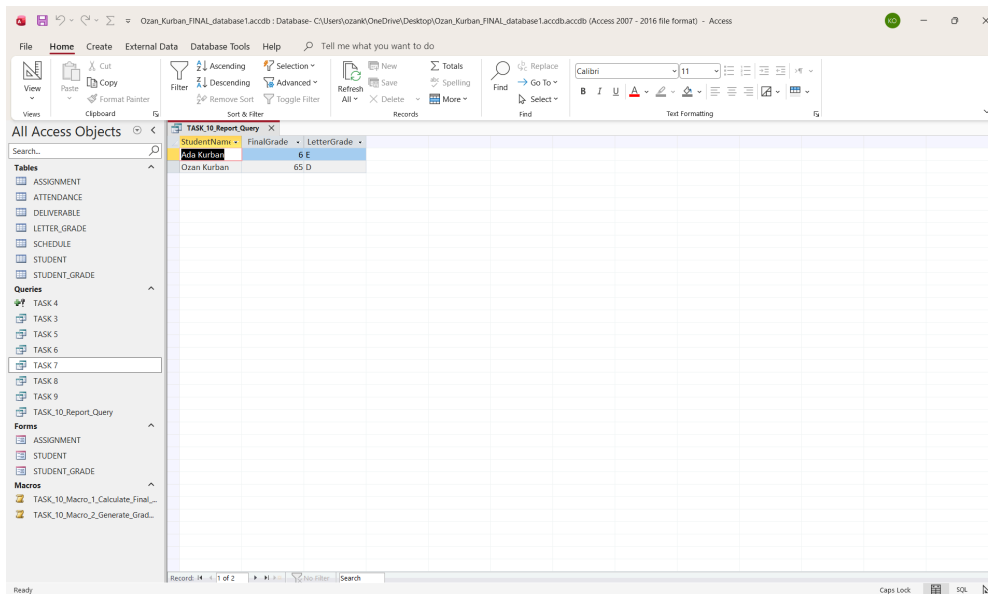
T6.FinalGrade,

T7.LetterGrade

FROM

[TASK 6] AS T6

INNER JOIN [TASK 7] AS T7 ON T6.StudentName = T7.StudentName;



Database Administration (DBA) Functions

Identify and describe three key DBA functions to maintain your grading database.

Examples

- **Concurrency Control:** Ensures accurate simultaneous access for multiple users.
- **Security Management:** Protects confidential student data through permissions and passwords.

- Backup and Recovery: Enables database restoration in case of data loss or corruption.

Submission

Provide 1–2 paragraphs per function describing Purpose, Implementation Details, and Expected Outcomes:

Concurrency control is vital because it ensures that when multiple users access or update the grading database at the same time, the stored data remains accurate and consistent. Without these proper controls, simultaneous edits could override each other, leading to incorrect grades or incomplete records. In Microsoft Access, this is typically handled using record-level locking, which locks only the specific record being edited rather than the entire table. This prevents conflicts when two users attempt to modify the same grade, and as a result, grade entries remain reliable, data conflicts are minimized, and the overall integrity of the database is maintained even in multi-user scenarios.

Furthermore, security management protects sensitive student information—such as names, grades, and attendance—from unauthorized access. Effective security ensures that only approved users, such as instructors or administrators, can view or modify academic records. Security in Access can be implemented through password protection, encryption, user-level permissions, and restricting write access to specific individuals. Additionally, storing the database in a protected network location adds an extra layer of security. These measures help prevent data leaks, accidental changes, or unauthorized manipulation of grades, ensuring the confidentiality and integrity of all student records.

Finally, backup and recovery protect the grading database against data loss caused by corruption, accidental deletions, hardware failures, or user mistakes. A reliable backup strategy ensures that all academic data can be restored quickly when needed. In practice, this involves creating regular copies of the .accdb file, storing backups on cloud platforms like OneDrive, or maintaining versioned local backups. Recovery procedures allow the instructor to restore the database to a previous state with minimal disruption. The expected outcome is greater resilience: even if something goes wrong, all student grades, attendance records, and deliverables can be recovered without losing work.

Final Reflection and Learning Outcomes

Reflect on what you have learned about database design, automation, and analysis.

Learning Outcomes

- Design and document a relational database (ERD and SQL schema)

- Query and analyze performance data using SQL
- Automate database processes with macros
- Apply DBA principles (security, backup, integrity)
- Use BI concepts for actionable insights

Reflection Questions (250–300 words)

- What was the most challenging part of designing and managing your database?
- How did SQL, ERDs, and macros change your understanding of data systems?
- How can these skills apply to real-world business or education settings?
- What new questions or ideas about data and BI emerged for you?
- What were the strengths and what were the weaknesses of this class?
- What would you suggest as improvements for future semesters?

Designing and managing this database was a transformative experience that highlighted the complexity of structuring raw data into meaningful information. The most challenging aspect was ensuring the logical integrity of the Entity Relationship Diagram (ERD), particularly defining the cardinality and participation constraints between the STUDENT, ATTENDANCE, and STUDENT_GRADE tables. Mapping out how a single student relates to multiple attendance records and various deliverables required a deep understanding of database normalization to prevent data redundancy while maintaining accuracy.

Working with SQL, ERDs, and macros fundamentally shifted my understanding of data systems from static spreadsheets to dynamic, interconnected environments. Using SQL to translate a conceptual design into a functional MS Access schema taught me how specific data types, like COUNTER for primary keys and YESNO for attendance, dictate system behavior. Furthermore, developing macros like Calculate_Final_And_Letter_Grades demonstrated that automation is not just a convenience but a necessity for maintaining real-time accuracy in any professional data environment. These skills are directly applicable to business and education settings, where automating performance reports and securing sensitive student or client data through DBA principles is a standard requirement.

Reflecting on the course, the primary strength was the instructor's highly effective teaching style; the material was presented with a positive and engaging energy that made complex database concepts much easier to grasp. However, I believe the curriculum could be even more impactful with a heavier focus on hands-on coding and advanced SQL syntax. Additionally, moving to an online format would better align with

the digital nature of the tools we use, such as Lucidchart and MS Access. These improvements would provide future students with a more streamlined and technically rigorous learning experience that mirrors the remote and code-heavy demands of the modern tech industry.